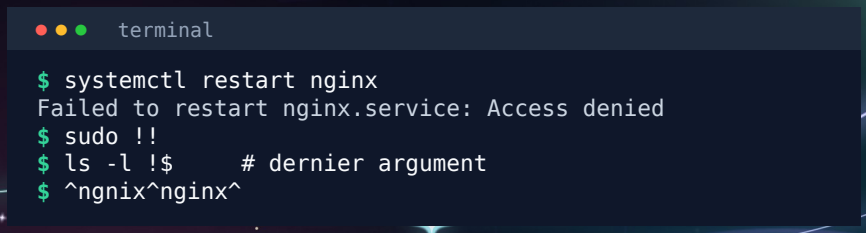
GUIDE PRATIQUE · ADMINISTRATION LINUX

Le « ! » de Bash en 15 usages

Historique, arguments, corrections et négation logique : le point d'exclamation sous toutes ses formes

!!, !$, ^a^b^, :p, [! -d], if !, !(motif), ${!var}… : 15 fiches pratiques testées sous Bash 5.2, les pièges à éviter et les réflexes pour l'utiliser sans risque.



RÉALISÉ PAR

Ngouye GNING

VERSION
1.0

DATE
Octobre 2026

CONTENU
15 fiches · 4 parties · annexes

NAVIGATION

Sommaire

Introduction 3

Le ! en un coup d'œil 5

Rappeler l'historique 8

01 !n — Réexécuter une commande par son numéro 9
02 !-n — Rappeler une commande par sa position 10
03 !! — Répéter la dernière commande 11
04 !chaîne — Rappeler par le début ou le contenu 12

Réutiliser et transformer 13

05 !$ · !^ · !* — Réutiliser les arguments précédents 14
06 !:n — Choisir un argument ou une plage 15
07 ^ancien^nouveau^ — Corriger une faute de frappe 16
08 :h · :t · :r · :e — Découper un chemin de fichier 17

Utiliser l'historique sans risque 18

09 :p — Prévisualiser avant d'exécuter 19
10 set +H — Éviter l'erreur « event not found » 20
11 histexpand — Historique et scripts : ce qui marche 21

La négation logique 22

12 [! -d rep] — Tester l'absence d'un fichier 23
13 if ! commande — Inverser un code de retour 24
14 !(motif) — Exclure des fichiers d'un motif 25
15 ${!var} — Expansion indirecte et clés de tableau 26

Conclusion 27

Annexes 28

A. Aide-mémoire : « Je veux… » 28
B. Référence de l'expansion d'historique 29
C. Réglages utiles de l'historique 29
D. Le ! ailleurs dans Linux 30
E. Messages d'erreur courants 30
F. Glossaire 31
G. Sources et références 31

AVANT-PROPOS

Introduction

La plupart des utilisateurs de Linux ne croisent le point d'exclamation que dans deux situations : le célèbre `sudo !!`, ou un mot de passe contenant un `!` qui déclenche l'erreur « event not found ». Ce simple caractère cache pourtant deux mécanismes puissants que beaucoup ignorent pendant des années.



La raison est que `!` joue **deux rôles totalement différents** dans Bash. Dans un terminal interactif, il déclenche l'**expansion de l'historique** : rappeler, réutiliser et corriger les commandes déjà tapées. Dans les tests, les commandes, les motifs de fichiers et l'expansion des paramètres, il exprime la **négation** — et fonctionne alors aussi bien en interactif que dans les scripts.

> COMMANDES RÉTABLIES ET ERREURS CORRIGÉES
>
> La traduction automatique de l'article d'origine a francisé du code : « histoire » pour `history`, « boutique -s extglob » pour `shopt -s extglob`, « ensemble +H » pour `set +H`, « si … alors … fi » pour `if … then … fi`, « déclarer -A » pour `declare -A`… Ce guide rétablit tout. Il corrige aussi plusieurs erreurs : `!:3*` désigne les arguments **à partir du troisième** (et non « 2 à 4 »), `:e` renvoie l'extension **avec** son point, l'option de prévisualisation s'appelle `histverify`, et l'exemple `chown … !*` reprenait par erreur l'option `-p`.

Objectifs du guide

- Rappeler n'importe quelle commande de l'historique **sans la retaper** ;
- Réutiliser et transformer les **arguments** des commandes précédentes ;
- Éviter les pièges : **prévisualiser** avant d'exécuter, corriger l'erreur « event not found » ;
- Maîtriser la **négation** dans les scripts : tests, codes de retour, motifs d'exclusion, expansion indirecte.

Public et environnement

Ce guide s'adresse aux administrateurs, développeurs et utilisateurs avancés du terminal. Tous les exemples ont été vérifiés avec **Bash 5.2**. zsh et ksh proposent une expansion d'historique proche ; `dash` et le `sh` POSIX ne la proposent pas du tout — une différence décisive dès que l'on écrit des scripts.

Conventions utilisées

NOTATION	SIGNIFICATION
`$ commande`	Commande tapée dans un terminal Bash interactif
Ligne qui suit un `!`	La commande développée, que Bash affiche avant de l'exécuter
`# texte`	Commentaire explicatif dans un bloc de code
Interactif Partout	Fonctionne seulement dans un terminal, ou aussi dans les scripts
Essentiel Pratique Avancé	Niveau de chaque fiche

> **À PROPOS DES EXEMPLES**
> Les numéros d'historique (1549, 1551…) et les chemins varient d'un système à l'autre : utilisez `history` pour trouver les vôtres. Les messages de Bash sont reproduits en anglais, tels qu'ils s'affichent.

SYNTHÈSE

Le ! en un coup d'œil

Un même caractère, deux mécanismes. L'**expansion de l'historique** agit à la lecture de la ligne, avant toute exécution, et seulement en interactif. La **négation** fait partie du langage et fonctionne partout.

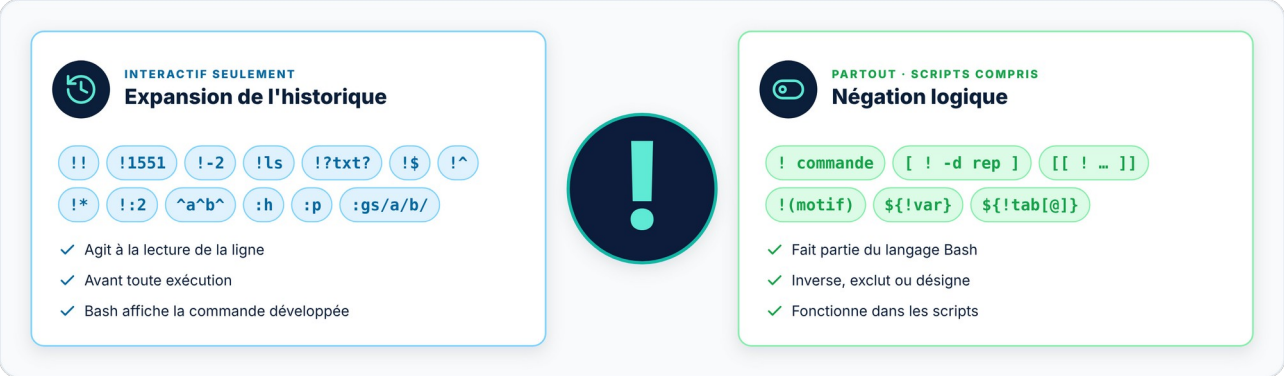


Figure 1 — *Les deux visages du ! dans Bash.*

> **COMMENT SAVOIR LEQUEL AGIT ?**
> Si `!` est suivi d'un chiffre, d'un `!`, d'un `$`, d'un `^`, d'un `*`, d'un `-`, d'un `?` ou d'un mot, c'est l'historique. S'il est suivi d'une espace, d'un `=` ou d'une parenthèse (motif étendu), ou s'il se trouve dans `${…}`, c'est le langage.

Anatomie d'une expansion d'historique

Une expansion d'historique se compose de trois parties, dont seule la première est obligatoire :

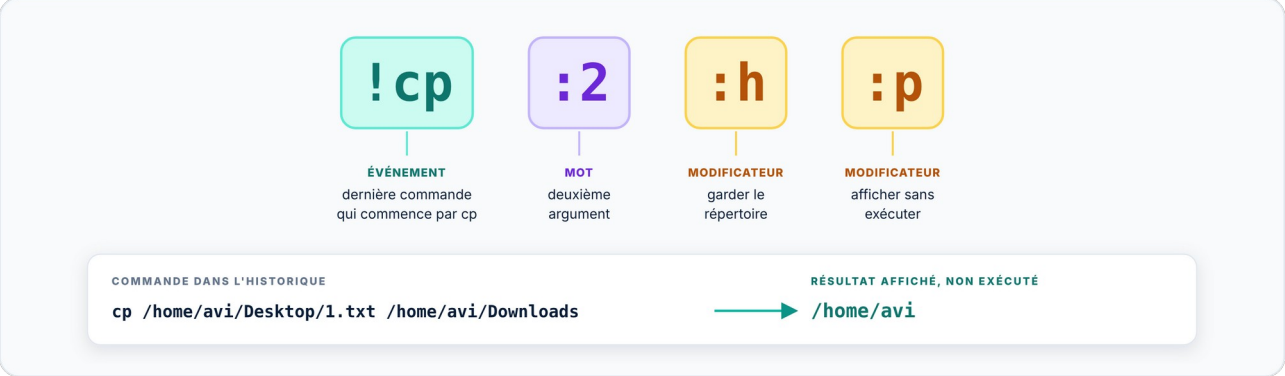


Figure 2 — *Événement, mot, modificateurs : la grammaire de l'expansion d'historique.*

PARTIE	RÔLE	EXEMPLES
Événement	Quelle commande de l'historique	`!!` `!1551` `!-2` `!ls` `!?Desktop?`
Mot	Quelle partie de cette commande (après `:`)	`:0` `:2` `:^` `:$` `:*` `:2-4` `:3*`
Modificateurs	Comment la transformer (chacun précédé de `:`)	`:h` `:t` `:r` `:e` `:p` `:gs/a/b/`

> **LIRE DE GAUCHE À DROITE**
> `!cp:2:h:p` se lit : « dans la dernière commande qui commence par `cp`, prends le deuxième argument, garde son répertoire, et affiche le résultat sans l'exécuter ».

Les 15 usages du guide

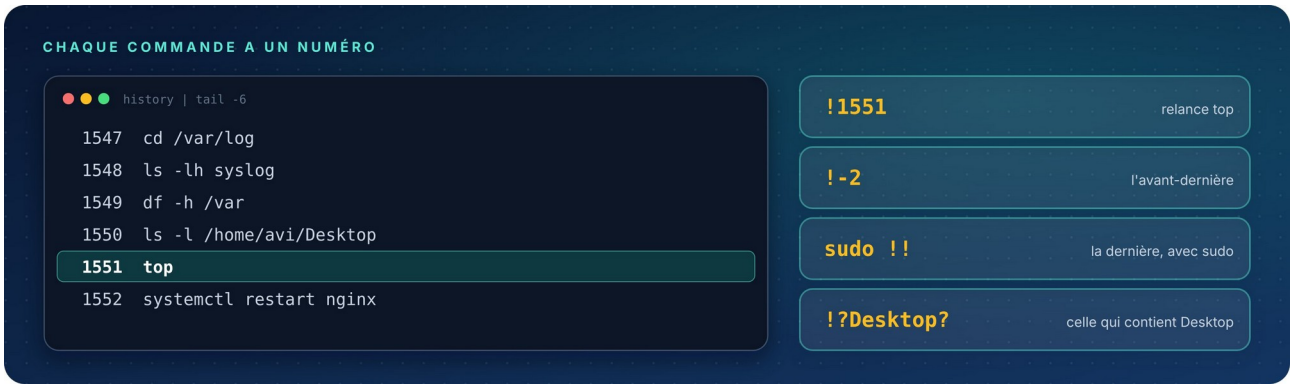
N°	SYNTAXE	RÔLE	OÙ ?	NIVEAU
01	`!1551`	Réexécuter une commande par son numéro	Interactif	Essentiel
02	`!-2`	Rappeler une commande par sa position	Interactif	Pratique
03	`!!`	Répéter la dernière commande (sudo !!)	Interactif	Essentiel
04	`!ls · !?txt?`	Rappeler par le début ou le contenu	Interactif	Pratique
05	`!$ · !^ · !*`	Réutiliser les arguments précédents	Interactif	Essentiel
06	`!:2 · !cp:2`	Choisir un argument ou une plage	Interactif	Pratique
07	`^a^b^`	Corriger une faute de frappe	Interactif	Essentiel
08	`:h · :t · :r · :e`	Découper un chemin de fichier	Interactif	Avancé
09	`:p`	Prévisualiser avant d'exécuter	Interactif	Essentiel
10	`set +H`	Éviter l'erreur « event not found »	Interactif	Pratique
11	`histexpand`	Historique et scripts : ce qui marche	Scripts	Pratique
12	`[ ! -d rep ]`	Tester l'absence d'un fichier	Partout	Essentiel
13	`if ! cmd`	Inverser un code de retour	Partout	Essentiel
14	`!(motif)`	Exclure des fichiers d'un motif	Partout	Avancé
15	`${!var}`	Expansion indirecte, clés de tableau	Partout	Avancé

Essentiel dès le premier jour Pratique usage régulier Avancé scripts et cas particuliers



Rappeler l’historique

Chaque commande tapée est mémorisée et numérotée. Ces quatre raccourcis la retrouvent et la relancent instantanément : par son numéro, par sa position, en entier, ou par un mot qu’elle contient.



DANS CETTE PARTIE

01	`!n`	Réexécuter une commande par son numéro
02	`!-n`	Rappeler une commande par sa position
03	`!!`	Répéter la dernière commande
04	`!chaîne`	Rappeler par le début ou le contenu

FICHE 01 · HISTORIQUE

01 !n — Réexécuter une commande par son numéro

OBJECTIF	CONTEXTE	NIVEAU
Relancer une commande précise sans la retaper	Terminal interactif	Essentiel

Bash numérote chaque commande enregistrée dans l'historique. Ce numéro s'appelle un **événement** : `!` suivi du numéro relance la commande correspondante. Bash affiche d'abord la commande développée, puis l'exécute.

```bash
$ history | tail -4
 1549  pwd
 1550  ls -l
 1551  top
 1552  df -h
$ !1551
top
```

PAS D'ESPACE APRÈS LE !

`!1551` déclenche l'expansion. `! 1551`, avec une espace, ne la déclenche pas : Bash cherche alors une commande nommée `1551` et répond « bash: 1551: command not found ».

Retrouver un numéro rapidement

```bash
$ history | grep nginx            # filtrer l'historique
 1498  sudo systemctl reload nginx
 1544  sudo nginx -t
$ history 5                       # seulement les 5 dernières commandes
```

UN NUMÉRO STABLE

Le numéro d'un événement ne change pas pendant la session : c'est la référence la plus fiable pour relancer plusieurs fois la même commande. Pour chercher par mot-clé de façon interactive, Ctrl+R (recherche inverse) reste l'alternative la plus confortable.

FICHE 02 · HISTORIQUE

02 !-n — Rappeler une commande par sa position

OBJECTIF	CONTEXTE	NIVEAU
Relancer l'une des dernières commandes	Terminal interactif	Pratique

Pas besoin de connaître le numéro : `!-1` désigne la dernière commande exécutée (comme `!!`), `!-2` l'avant-dernière, `!-3` celle d'avant, et ainsi de suite.



Figure 3 — *Numéros absolus et positions relatives dans l'historique.*

```bash
$ cd /var/log
$ ls -lh syslog
$ df -h /var
$ !-3                          # trois commandes en arrière
cd /var/log
```

DES RÉFÉRENCES QUI BOUGENT

Une position relative est recalculée à chaque nouvelle commande : `!-3` ne désigne plus la même ligne une commande plus tard. Pour répéter plusieurs fois la même commande, préférez son numéro (fiche 01). En cas de doute, ajoutez `:p` pour afficher sans exécuter (fiche 09).

FICHE 03 · HISTORIQUE

03 !! — Répéter la dernière commande

OBJECTIF	CONTEXTE	NIVEAU
Relancer, compléter ou élever les privilèges	Terminal interactif	Essentiel

`!!` se développe en la **commande précédente entière**. Son usage le plus célèbre : relancer avec `sudo` une commande refusée faute de privilèges.

bash

```
$ systemctl restart nginx
Failed to restart nginx.service: Access denied
See system logs and 'systemctl status nginx.service' for details.
$ sudo !!
sudo systemctl restart nginx
```

Comme `!!` reprend toute la ligne, on peut la compléter, par exemple avec une redirection :

bash

```
$ ip -4 -o addr show | awk '{print $4}' | cut -d/ -f1
127.0.0.1
192.168.0.10
$ !! > ip.txt
ip -4 -o addr show | awk '{print $4}' | cut -d/ -f1 > ip.txt
```

Sans `sudo`, avec `su` : les guillemets **doubles** sont indispensables, car l'expansion d'historique n'a pas lieu entre guillemets simples.

bash

```
$ su -c "!!" root
su -c "systemctl restart nginx" root
Password:
```

SUDO N'EST PAS UNE SOLUTION UNIVERSELLE

`sudo !!` ne corrige que les erreurs de **droits**. Si Bash répond « command not found » — par exemple pour `ifconfig` —, c'est le paquet qui manque : `sudo apt install net-tools` (Debian, Ubuntu) ou `sudo dnf install net-tools` (RHEL, Rocky Linux, Fedora). Mieux encore : utilisez la commande moderne `ip addr`.

FICHE 04 · HISTORIQUE

04 !chaîne — Rappeler par le début ou le contenu

OBJECTIF	CONTEXTE	NIVEAU
Retrouver la dernière commande qui correspond	Terminal interactif	Pratique

`!ls` relance la **plus récente** commande qui **commence** par `ls`. `!?Desktop?` relance la plus récente qui **contient** `Desktop`, n'importe où dans la ligne.

```bash
$ ls /home > /dev/null
$ ls -l /home/avi/Desktop > /dev/null
$ ls -lA /usr/bin > /dev/null
$ !ls
ls -lA /usr/bin > /dev/null
$ !?Desktop?
ls -l /home/avi/Desktop > /dev/null
```

Le piège classique : le texte qui suit l'événement n'est **pas** un critère de recherche. Il est simplement **ajouté** à la commande retrouvée :

```bash
$ !ls -l
ls -lA /usr/bin > /dev/null -l
```

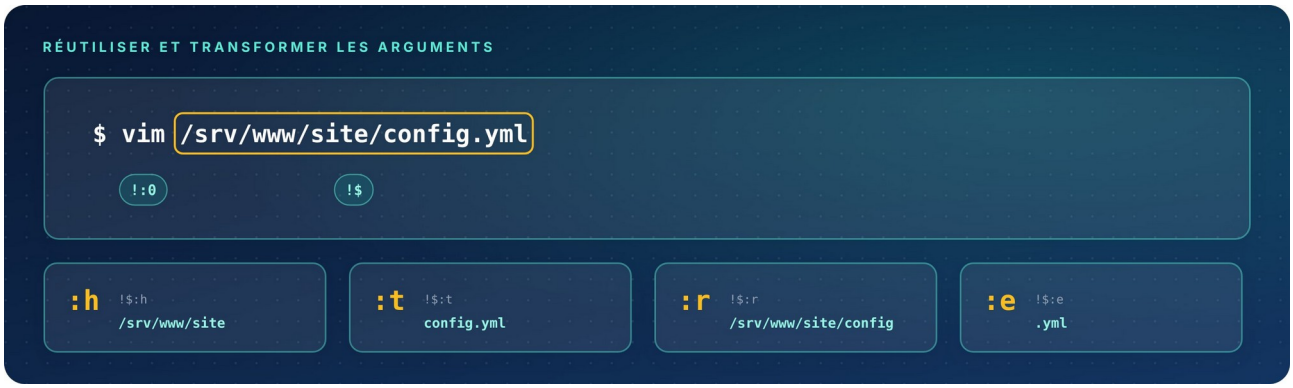
SYNTAXE	RELANCE LA DERNIÈRE COMMANDE QUI…
`!chaîne`	commence par chaîne
`!?chaîne?`	contient chaîne (le `?` final est facultatif en fin de ligne)

> **UNE SEULE CORRESPONDANCE**
> Bash exécute toujours la correspondance la plus récente, sans proposer de choix. Ajoutez `:p` (`!ls:p`, `!?Desktop?:p`) pour vérifier avant d'exécuter, ou utilisez Ctrl+R pour parcourir les correspondances une à une. Si rien ne correspond, Bash répond « event not found ».



Réutiliser et transformer

Les arguments des commandes précédentes — chemins longs, listes de fichiers — sont réutilisables à la demande : en entier, un par un, ou transformés. Et une faute de frappe se corrige en une ligne.



DANS CETTE PARTIE

05	`!$ · !^ · !*`	Réutiliser les arguments précédents
06	`!:n`	Choisir un argument ou une plage
07	`^ancien^nouveau^`	Corriger une faute de frappe
08	`:h · :t · :r · :e`	Découper un chemin de fichier

FICHE 05 · ARGUMENTS

05 !$ · !^ · !* — Réutiliser les arguments précédents

OBJECTIF	CONTEXTE	NIVEAU
Ne plus retaper les chemins longs	Terminal interactif	Essentiel

Vous venez de lister un répertoire et voulez maintenant le format long ? Inutile de retaper le chemin : `!$` insère le **dernier argument** de la commande précédente.

bash

```
$ ls /home/alice/Binary/firefox
$ ls -l !$
ls -l /home/alice/Binary/firefox
```

DÉSIGNATEUR	MOT(S) REPRIS DE LA COMMANDE PRÉCÉDENTE
`!$`	Le dernier argument
`!^`	Le premier argument
`!*`	Tous les arguments, sans le nom de la commande
`!:0`	Le nom de la commande seul

ATTENTION AUX OPTIONS AVEC !*

`!*` reprend **aussi les options**. Après `mkdir -p app/logs app/cache app/tmp`, la commande `chmod 750 !*` devient `chmod 750 -p app/logs …` et échoue. `!:2*` ne garde que les mots à partir du deuxième — ici, les seuls répertoires. Après l'échec, le `mkdir` est devenu l'avant-dernière commande : on écrit donc `!-2:2*`.

bash

```
$ mkdir -p app/logs app/cache app/tmp
$ chmod 750 !*
chmod 750 -p app/logs app/cache app/tmp
chmod: invalid option -- 'p'
$ chmod 750 !-2:2*              # mots 2 et suivants du mkdir
chmod 750 app/logs app/cache app/tmp
```

LE RACCOURCI CLAVIER ALT+.

Alt+. (ou Échap puis .) insère le dernier argument de la commande précédente à l'emplacement du curseur, **sans rien exécuter** : vous voyez ce que vous obtenez. Chaque nouvel appui remonte d'une commande dans l'historique.

FICHE 06 · ARGUMENTS

06 !:n — Choisir un argument ou une plage

OBJECTIF	CONTEXTE	NIVEAU
Extraire exactement la partie utile	Terminal interactif	Pratique

Les **désignateurs de mots** sélectionnent un mot par sa position. Le mot 0 est le nom de la commande, le mot 1 le premier argument, et ainsi de suite.

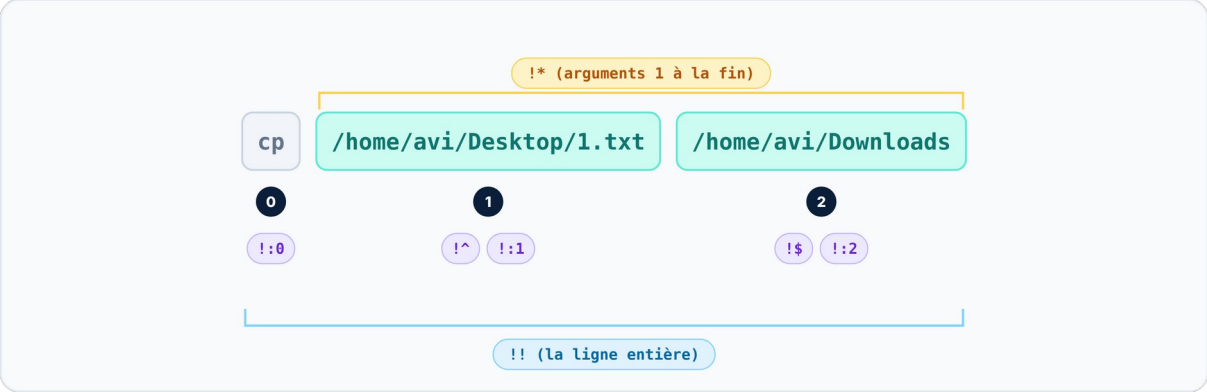


Figure 4 — *Les positions des mots d'une commande et leurs désignateurs.*

bash

```
$ cp /home/avi/Desktop/1.txt /home/avi/Downloads
$ echo "Source : !^"
echo "Source : /home/avi/Desktop/1.txt"
Source : /home/avi/Desktop/1.txt
$ echo "Destination : !cp:2"
echo "Destination : /home/avi/Downloads"
Destination : /home/avi/Downloads
```

DÉSIGNATEUR	MOT(S) REPRIS
`!:2`	Le deuxième argument de la commande précédente
`!:2-4`	Les arguments 2 à 4
`!:3*`	Les arguments 3 jusqu'au dernier (comme !:3-$)
`!:2-`	Les arguments 2 jusqu'à l'avant-dernier
`!cp:2`	Le deuxième argument de la dernière commande commençant par cp

CORRECTION

L'article d'origine indiquait que `!:3*` réutilise « les arguments 2 à 4 » : c'est inexact. `!:3*` prend tous les arguments **à partir du troisième**. Si la commande en compte moins de trois, Bash répond « bad word specifier ».

FICHE 07 · ARGUMENTS

07 ^ancien^nouveau^ — Corriger une faute de frappe

OBJECTIF	CONTEXTE	NIVEAU
Remplacer un mot dans la commande précédente	Terminal interactif	Essentiel

La **substitution rapide** remplace la première occurrence d'un texte dans la commande précédente, puis exécute le résultat.

```bash
$ systemctl status ngnix
Unit ngnix.service could not be found.
$ ^ngnix^nginx^
systemctl status nginx
```

Seule la **première** occurrence change. Pour remplacer **toutes** les occurrences, utilisez le modificateur `:gs` (*global substitute*) :

```bash
$ vim /etc/nginx/nginx.conf
$ ^nginx^apache^                    # première occurrence seulement
vim /etc/apache/nginx.conf
$ !-2:gs/nginx/apache/              # toutes les occurrences
vim /etc/apache/apache.conf
```

FORME	EFFET
`^a^b^`	Remplace la première occurrence de a par b dans la commande précédente
`!!:s/a/b/`	Forme explicite équivalente
`!!:gs/a/b/`	Remplace toutes les occurrences
`!!:gs/a/b/:p`	Affiche le résultat sans l'exécuter

> **BON À SAVOIR**
> Le `^` final est facultatif : `^ngnix^nginx` fonctionne aussi. Le séparateur `/` de `:s` peut être remplacé par un autre caractère quand le texte contient des barres obliques : `!!:gs|/var|/srv|`.

FICHE 08 · ARGUMENTS

08 :h · :t · :r · :e — Découper un chemin de fichier

OBJECTIF	CONTEXTE	NIVEAU
Extraire répertoire, nom ou extension	Terminal interactif	Avancé

Les **modificateurs** transforment le mot rappelé. Dans le feu de l'action, ils remplacent avantageusement `dirname` et `basename`.



Figure 5 — *Ce que renvoie chaque modificateur pour /etc/nginx/nginx.conf.*

```bash
$ ls -l /etc/nginx/nginx.conf
$ echo !$:h !$:t !$:r !$:e
echo /etc/nginx nginx.conf /etc/nginx/nginx .conf
/etc/nginx nginx.conf /etc/nginx/nginx .conf
```

Une combinaison très utile : modifier un fichier, puis se placer dans son répertoire.

```bash
$ vim /srv/www/example.com/config/settings.yml
$ cd !$:h
cd /srv/www/example.com/config
```

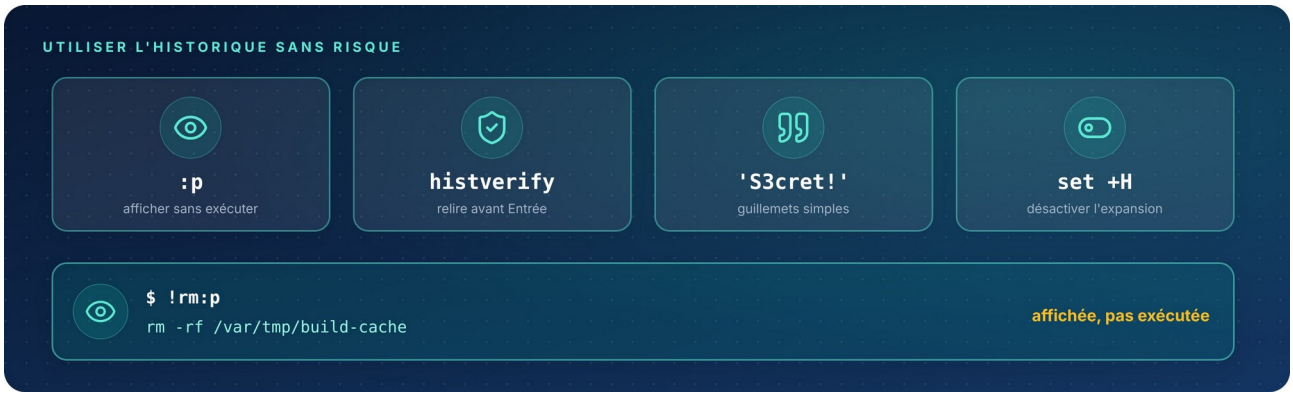
COMBINAISONS ET CORRECTION

Les modificateurs se combinent avec n'importe quel événement : `!!:$:t` donne le nom de fichier du dernier argument de la commande précédente, `!1551:$:h` le répertoire du dernier argument de l'événement 1551. L'article d'origine annonçait `:e` → « conf » : Bash conserve en réalité le point (`.conf`).



Utiliser l’historique sans risque

L’expansion d’historique s’exécute immédiatement : un `!rm` mal ciblé peut coûter cher. Trois réflexes rendent son usage sûr : prévisualiser, protéger les textes qui contiennent un `!`, et ne jamais compter sur elle dans un script.



DANS CETTE PARTIE

09	`:p`	Prévisualiser avant d’exécuter
10	`set +H`	Éviter l’erreur « event not found »
11	`histexpand`	Historique et scripts : ce qui marche

FICHE 09 · SÉCURITÉ

09 :p — Prévisualiser avant d'exécuter

OBJECTIF	CONTEXTE	NIVEAU
Vérifier la commande développée sans risque	Terminal interactif	Essentiel

Le modificateur `:p` (*print*) affiche la commande développée **sans l'exécuter**. Elle est en même temps ajoutée à l'historique : `!!` suffit ensuite pour la lancer, ou la flèche haut pour la modifier.

bash

```
$ !rm:p
rm -rf /var/tmp/build-cache
$ !!                              # exécuter, après vérification
rm -rf /var/tmp/build-cache
```

`:p` fonctionne avec toutes les formes : `!!:p`, `!1551:p`, `!?Desktop?:p`, `!!:gs/a/b/:p`.

Toujours relire : l'option histverify

Avec `histverify`, Bash ne lance plus directement une commande issue de l'historique : il la place sur la ligne de saisie, où vous pouvez la relire, la modifier ou l'annuler avant d'appuyer sur Entrée.

bash

```
$ shopt -s histverify
$ echo 'shopt -s histverify' >> ~/.bashrc       # réglage permanent
```

> **LE BON RÉFLEXE**
> Activez `histverify` sur les serveurs de production : vous gardez la rapidité de `!!` et `!$`, sans exécution surprise. Et prenez l'habitude de `:p` avant tout `!rm`, `!dd` ou commande destructrice rappelée de l'historique.

FICHE 10 · SÉCURITÉ

10 set +H — Éviter l'erreur « event not found »

OBJECTIF	CONTEXTE	NIVEAU
Utiliser un ! littéral dans une commande	Terminal interactif	Pratique

Un `!` placé entre **guillemets doubles** est interprété comme une expansion d'historique. Un mot de passe, une URL ou un message qui en contient déclenche alors une erreur :

```bash
$ mysql -u admin -p"S3cret!Pass"
bash: !Pass: event not found
```

SOLUTION	EXEMPLE ET REMARQUE
Guillemets simples	`mysql -u admin -p'S3cret!Pass'` : tout est littéral, la solution la plus sûre
Barre oblique hors guillemets	`echo Bravo\!` affiche `Bravo!`
Désactiver l'expansion	`set +H` (ou `set +o histexpand`), puis `set -H` pour la rétablir

```bash
$ echo "Bravo\!"                    # piège : la barre oblique reste affichée
Bravo\!
$ echo "Bonjour !"                   # ! suivi d'un guillemet : aucune expansion
Bonjour !
```

ET POUR LES MOTS DE PASSE

Évitez de taper un mot de passe en clair sur la ligne de commande : il reste dans `~/.bash_history` et apparaît dans la liste des processus. Préférez `mysql -u admin -p` sans valeur, qui le demande de façon masquée, ou un fichier d'options protégé.

FICHE 11 · SÉCURITÉ

11 histexpand — Historique et scripts : ce qui marche

OBJECTIF	CONTEXTE	NIVEAU
Distinguer interactif et script	Scripts shell	Pratique

L'expansion d'historique est réservée aux shells **interactifs**. Dans un script, l'option `histexpand` est désactivée et aucun historique n'est chargé : `!!`, `!$`, `!123`, `!chaîne` ou `^a^b^` y restent du texte ordinaire.

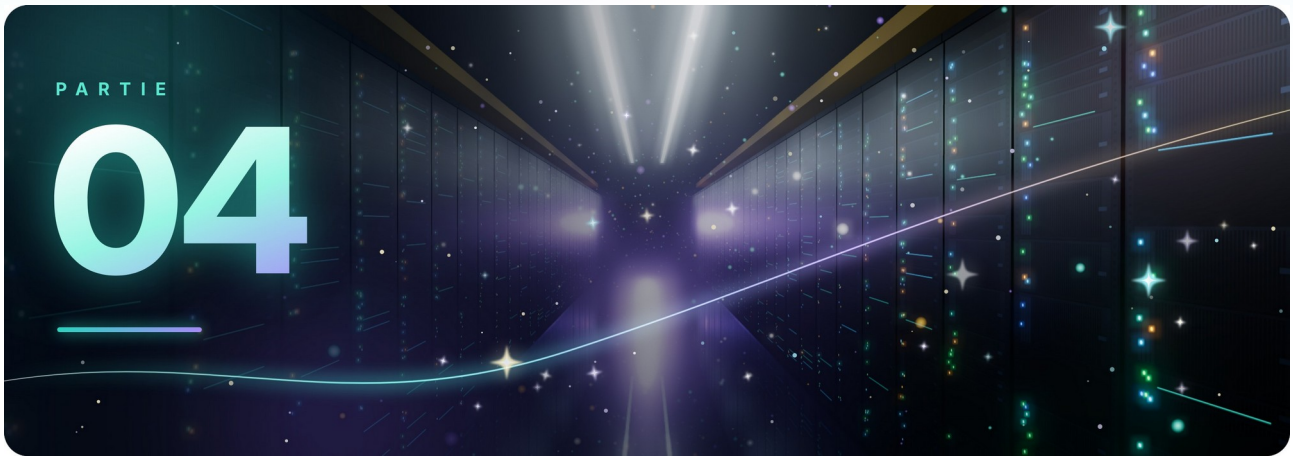
bash

```
$ bash -c 'set -o | grep histexpand'
histexpand      off
$ echo $-                # dans un terminal : la lettre H signale histexpand
himBHs
```

INTERACTIF SEULEMENT	PARTOUT, SCRIPTS COMPRIS
`!!` `!$` `!^` `!*`	`! commande` : inverser un code de retour
`!123` `!-2` `!chaîne` `!?texte?`	`[ ! -d rep ]` et `[[ ! … ]]`
`^ancien^nouveau^`	`!(motif)` avec `shopt -s extglob`
`:h` `:t` `:r` `:e` `:p` `:gs/a/b/`	`${!var}` et `${!tableau[@]}`

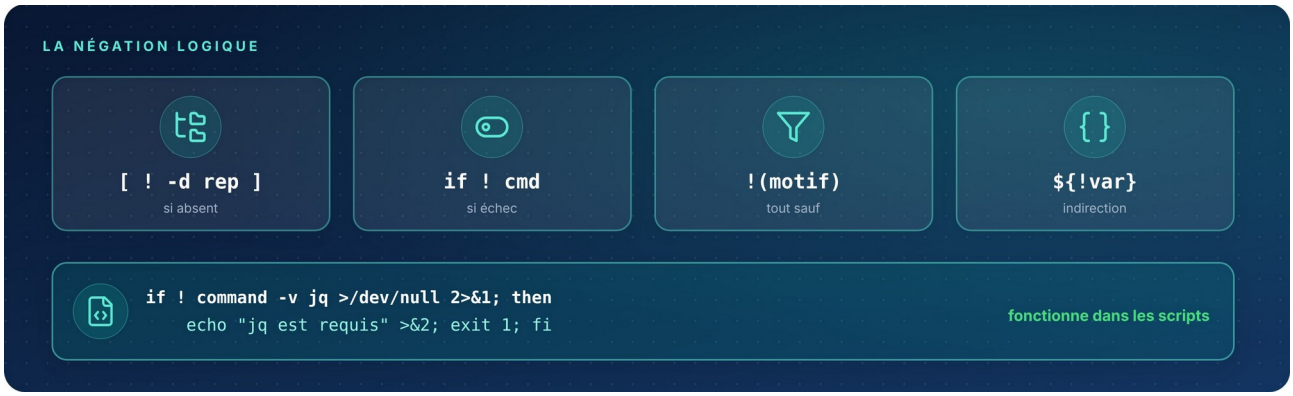
DANS UN SCRIPT, DES VARIABLES

Remplacez les raccourcis d'historique par des variables : `fichier=/etc/nginx/nginx.conf`, puis `dirname "$fichier"` et `basename "$fichier"`, ou les expansions `${fichier%/*}` (répertoire) et `${fichier##*/}` (nom).



La négation logique

Dans les tests, les commandes, les motifs de fichiers et les variables, `!` change de nature : il inverse, il exclut, il désigne. Cette fois, tout fonctionne aussi dans les scripts.



DANS CETTE PARTIE

12	`[ ! -d rep ]`	Tester l'absence d'un fichier
13	`if ! commande`	Inverser un code de retour
14	`!(motif)`	Exclure des fichiers d'un motif
15	`${!var}`	Expansion indirecte et clés de tableau

FICHE 12 · NÉGATION

12 [! -d rep] — Tester l'absence d'un fichier

OBJECTIF	CONTEXTE	NIVEAU
Agir seulement si quelque chose manque	Terminal et scripts	Essentiel

Dans un test, `!` inverse le résultat : `[ -d rep ]` est vrai si le répertoire existe, `[ ! -d rep ]` s'il **n'existe pas**.

verifier.sh

```
#!/bin/bash
if [ ! -d /srv/backups ]; then
    echo "Répertoire de sauvegarde absent : arrêt." >&2
    exit 1
fi
```

bash

```
$ [ ! -d /srv/backups ] && echo "absent" || echo "présent"
présent
```

TEST	VRAI SI… (ET AVEC ! : SI CE N'EST PAS LE CAS)
`-d chemin`	le chemin est un répertoire
`-f chemin`	le chemin est un fichier ordinaire
`-e chemin`	le chemin existe, quel que soit son type
`-x chemin`	le fichier est exécutable
`-s chemin`	le fichier existe et n'est pas vide
`-z "$v"`	la chaîne est vide

LE PIÈGE DE A && B || C

Ce n'est pas un vrai « si… sinon » : si `B` échoue, `C` s'exécute aussi. Pour des actions importantes, écrivez un `if … then … else … fi`. Et pour créer un répertoire, `mkdir -p` suffit : il ne fait rien si le répertoire existe déjà.

FICHE 13 · NÉGATION

13 if ! commande — Inverser un code de retour

OBJECTIF	CONTEXTE	NIVEAU
Réagir à un échec de façon lisible	Terminal et scripts	Essentiel

Placé devant une commande, `!` inverse son code de retour : un succès (0) devient un échec, un échec devient un succès. C'est la façon la plus lisible d'écrire « si ceci ne fonctionne pas, alors… ».

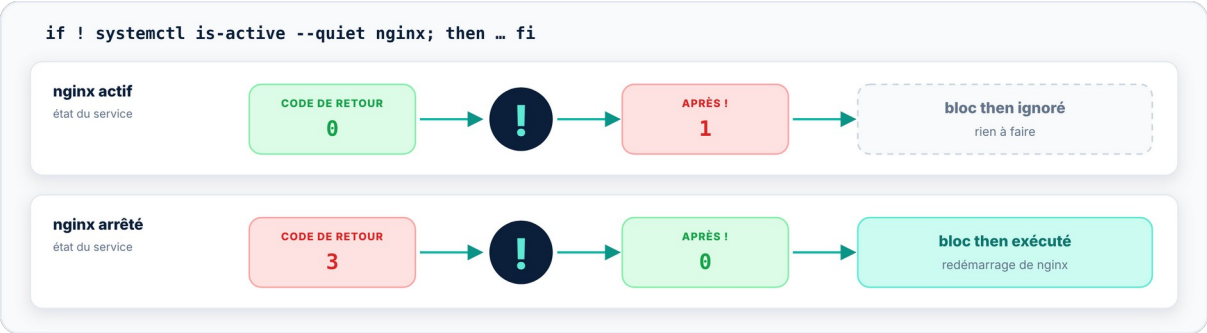


Figure 6 — *! inverse le code de retour avant que if ne le lise.*

surveiller.sh

```
#!/bin/bash
if ! systemctl is-active --quiet nginx; then
    echo "nginx est arrêté : redémarrage..."
    systemctl restart nginx
fi

if ! command -v jq >/dev/null 2>&1; then
    echo "jq est requis mais n'est pas installé." >&2
    exit 1
fi
```

AVEC SET -E

Une commande précédée de `!` ne provoque jamais l'arrêt d'un script lancé avec `set -e`, même si elle échoue. C'est voulu, mais à garder en tête lors du débogage.

FICHE 14 · NÉGATION

14 !(motif) — Exclure des fichiers d'un motif

OBJECTIF	CONTEXTE	NIVEAU
Agir sur tout, sauf certains fichiers	Terminal et scripts	Avancé

`!(motif)` fait partie des **motifs étendus** (*extglob*) : il correspond à tous les noms **sauf** ceux du motif. L'option est désactivée par défaut dans les scripts ; en interactif, certaines distributions l'activent déjà via bash-completion.

```bash
$ shopt extglob
extglob         off
$ shopt -s extglob
$ ls
README.md  a.pdf  b.txt  c.conf
$ ls !(*.pdf)                          # vérifier d'abord
README.md  b.txt  c.conf
$ rm !(*.pdf|*.conf|README.md)          # tout sauf PDF, .conf et README.md
```

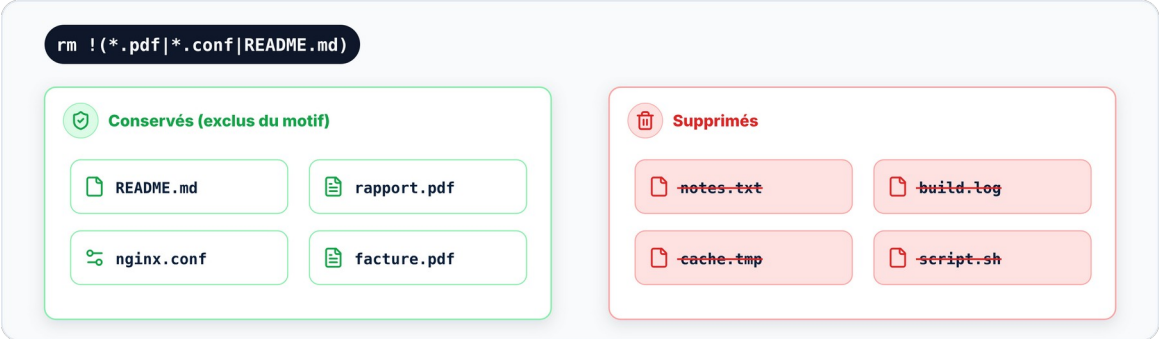


Figure 7 — *Ce que garde et ce que supprime rm !(*.pdf|*.conf|README.md).*

MOTIF ÉTENDU	CORRESPOND À
`!(motif)`	tout sauf le motif
`?(motif)`	zéro ou une occurrence du motif
`*(motif)` · `+(motif)`	zéro ou plusieurs · une ou plusieurs occurrences
`@(a|b)`	exactement l'un des motifs

VÉRIFIER AVANT DE SUPPRIMER

Testez toujours le motif avec `ls` avant `rm`. Les fichiers cachés ne sont pas concernés (sauf avec `shopt -s dotglob`) et le motif ne descend pas dans les sous-répertoires. Sans extglob, Bash répond « syntax error near unexpected token ».

FICHE 15 · NÉGATION

15 ${!var} — Expansion indirecte et clés de tableau

OBJECTIF	CONTEXTE	NIVEAU
Lire une variable dont le nom est dans une autre	Terminal et scripts	Avancé

Dans une expansion de paramètre, `!` a encore un autre sens : `${!nom}` utilise la **valeur** de `nom` comme **nom** d'une autre variable. Avec un tableau, `${!tab[@]}` renvoie les **indices** (ou les clés), pas les valeurs.

bash

```
$ nginx_port=8080
$ service=nginx_port
$ echo "${!service}"
8080
$ fruits=(pomme banane mangue)
$ echo "${!fruits[@]}"               # les indices, pas les valeurs
0 1 2
$ echo "${!BASH_V*}"                 # noms de variables commençant par BASH_V
BASH_VERSINFO BASH_VERSION
```

ports.sh

```
declare -A ports=([nginx]=80 [postgres]=5432 [redis]=6379)
for service in "${!ports[@]}"; do
    echo "$service écoute sur le port ${ports[$service]}"
done
```

bash

```
$ bash ports.sh
nginx écoute sur le port 80
redis écoute sur le port 6379
postgres écoute sur le port 5432
```

BON À SAVOIR

L'ordre des clés d'un tableau associatif n'est pas garanti. Depuis Bash 4.3, `declare -n` (référence nommée) offre une alternative plus lisible à `${!var}` : `declare -n ref=nginx_port; echo "$ref"`.

POUR CONCLURE

Conclusion

Le point d'exclamation de Bash mène une double vie : raccourci d'historique dans le terminal, opérateur de négation dans le langage. Savoir lequel des deux est à l'œuvre, c'est éviter la plupart des surprises — et gagner un temps considérable.

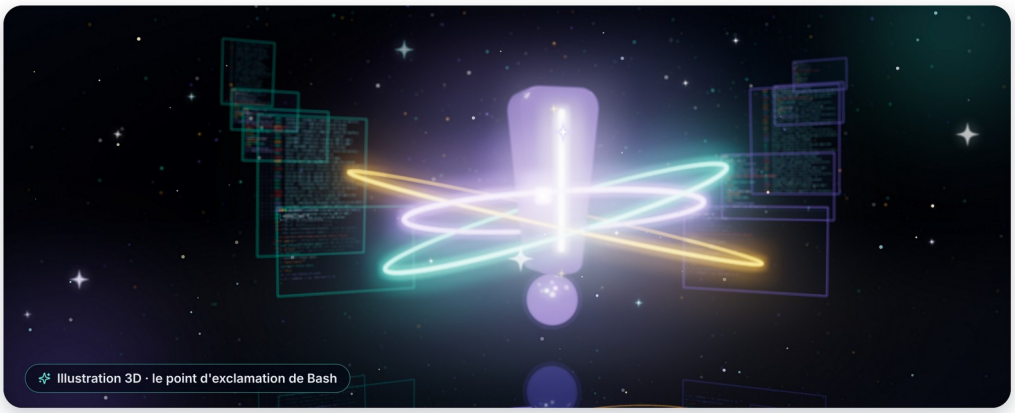




Figure 8 — *Le cycle sûr d'une commande rappelée de l'historique.*

Les réflexes à retenir

- `sudo !!` pour une erreur de droits — pas pour une commande introuvable ;
- `!$` et **Alt+.** pour les chemins ; `!:2*` plutôt que `!*` quand la commande a des options ;
- `:p` ou `histverify` avant tout `!rm` ou commande destructrice ;
- **Guillemets simples** pour tout texte qui contient un `!` ;
- **Pas d'expansion d'historique dans un script** : des variables à la place ;
- `ls !(motif)` avant `rm !(motif)`.

> POUR ALLER PLUS LOIN
>
> `fc` ouvre la dernière commande dans votre éditeur ; `history -d N` supprime une entrée ; la variable `HISTCONTROL=ignoreboth` évite d'enregistrer les doublons et les commandes précédées d'une espace.

RESSOURCES

Annexes

Ces annexes rassemblent les informations à garder sous la main : aide-mémoire, référence complète de l'expansion d'historique, réglages utiles, le ! ailleurs dans Linux, messages d'erreur et glossaire.

A. Aide-mémoire : « Je veux… »

JE VEUX…	COMMANDE
Relancer la dernière commande avec sudo	`sudo !!`
Relancer la commande n° 1551	`!1551`
Relancer l'avant-dernière commande	`!-2`
Relancer la dernière commande ssh	`!ssh`
Réutiliser le dernier argument	`!$` (ou Alt+.)
Réutiliser tous les arguments	`!*`
Corriger une faute de frappe	`^ngnix^nginx^`
Remplacer partout dans la commande	`!!:gs/ancien/nouveau/`
Aller dans le répertoire du fichier	`cd !$:h`
Voir sans exécuter	`!rm:p`
Écrire un ! littéral	`'texte!'` ou `set +H`
Tout supprimer sauf les PDF	`shopt -s extglob; rm !(*.pdf)`

B. Référence de l'expansion d'historique

TYPE	FORME	SIGNIFICATION
Événement	`!!`	La commande précédente (comme !-1)
Événement	`!n · !-n`	La commande numéro n · n commandes en arrière
Événement	`!chaîne · !?chaîne?`	La dernière commande qui commence par · contient chaîne
Événement	`!#`	La ligne en cours de saisie, jusqu'ici
Raccourci	`^a^b^`	La commande précédente, a remplacé par b
Mot	`0 · n · ^ · $`	Commande · argument n · premier · dernier
Mot	`x-y · x* · x-`	De x à y · de x à la fin · de x à l'avant-dernier
Mot	`*`	Tous les arguments (comme 1*)
Modificateur	`:h · :t`	Répertoire · nom (dirname · basename)
Modificateur	`:r · :e`	Sans extension · extension seule
Modificateur	`:p`	Afficher sans exécuter
Modificateur	`:s/a/b/ · :gs/a/b/`	Remplacer la première · toutes les occurrences
Modificateur	`:q`	Mettre le résultat entre guillemets simples

C. Réglages utiles de l'historique

~/.bashrc

```
HISTSIZE=10000                  # commandes gardées en mémoire
HISTFILESIZE=20000              # lignes conservées dans ~/.bash_history
HISTCONTROL=ignoreboth          # ni doublons, ni commandes précédées d'une espace
HISTTIMEFORMAT='%F %T  '        # horodatage dans la sortie de history
shopt -s histappend             # ajouter au fichier au lieu de l'écraser
shopt -s histverify             # relire avant d'exécuter
```

D. Le ! ailleurs dans Linux

SYNTAXE	RÔLE
`#!/bin/bash`	Shebang : l'interpréteur qui exécute le script
`[ "$a" != "$b" ]`	Comparaison « différent de »
`find . ! -name '*.log'`	Négation d'un critère de find
`awk '!vu[$0]++' fichier`	Supprimer les doublons en gardant l'ordre
`:w !sudo tee %`	Vim : enregistrer un fichier protégé
`ip route | grep -v default`	« Tout sauf » avec grep… sans aucun !

E. Messages d'erreur courants

MESSAGE	CAUSE ET SOLUTION
`bash: !Pass: event not found`	Un ! entre guillemets doubles : guillemets simples ou `set +H`
`bash: !zzz: event not found`	Aucune commande de l'historique ne correspond
`bash: :3*: bad word specifier`	La commande précédente a moins d'arguments que demandé
`bash: 1551: command not found`	Une espace entre ! et le numéro : écrire `!1551`
`chmod: invalid option -- 'p'`	`!*` a repris une option : utiliser `!:2*`
`syntax error near unexpected token`	Motif `!(…)` sans extglob : `shopt -s extglob`

F. Glossaire

TERME	DÉFINITION
Expansion d'historique	Remplacement d'une référence (!!, !$…) par du texte de l'historique, avant l'exécution
Événement	Une commande de l'historique, désignée par numéro, position ou texte
Désignateur de mot	Sélection d'un ou plusieurs mots d'un événement (:2, $, *)
Modificateur	Transformation appliquée au résultat (:h, :t, :p, :gs…)
histexpand	Option qui active l'expansion d'historique (set -H / set +H)
histverify	Option qui place la commande développée sur la ligne au lieu de l'exécuter
extglob	Option des motifs étendus : !(…), ?(…), *(…), +(…), @(…)
Code de retour	Nombre renvoyé par une commande : 0 = succès, autre = échec
Expansion indirecte	${!nom} : la valeur de nom sert de nom de variable
Shell interactif	Shell qui lit les commandes au clavier, avec historique et invite

G. Sources et références

- Avishek, « 15 exemples d'opérateurs Bash (!) que tout utilisateur Linux devrait connaître », Tecmint (tecmint.com), mis à jour le 4 août 2026. Article dont ce guide reprend les exemples, qu'il corrige et enrichit.
- GNU Bash Reference Manual : « History Interaction », « Pattern Matching », « Shell Parameter Expansion » ; page de manuel `bash(1)`, section HISTORY EXPANSION.
- Exemples vérifiés avec GNU Bash 5.2.

DOCUMENT RÉDIGÉ ET MIS EN FORME PAR

Ngouye GNING

Version 1.0 · Octobre 2026